Mersenne Exponents in the Shell–Leg Lattice: A Deterministic Construction via Binary Patterns

Martin Doina
github.com/gatanegro
ORCID: 0009-0002-3855-2268

March 23, 2026

Abstract

We present a deterministic construction of all 52 known Mersenne exponents (OEIS A000043) using a binary growth process from seeds 2 and 3. Each exponent is generated by a unique pattern of two operations: *shell* ($n \mapsto 2n$, digit 0) and *leg* ($n \mapsto 2n+1$, digit 1). The patterns form a tree structure where exponents cluster by shared prefixes, with only 50 intersection nodes among 579 total vertices. A *flip rule* eliminates duplicates, ensuring each path is unique. This provides a constructive characterization of Mersenne exponents independent of primality testing of $2^p - 1$. Complete enumeration of all patterns up to length 26 yields exactly the known exponents, suggesting the method captures the complete set. Interactive tools and source code are available at github.com/gatanegro/MERSENNE-COLLATZ.

2020 Mathematics Subject Classification. Primary 11A41; Secondary 11N05, 11Y11, 11B83.

Key words and phrases. Mersenne primes, Mersenne exponents, deterministic exponent construction, lattice branching.

1 Introduction

The Mersenne exponents p (primes such that $2^p - 1$ is prime) have fascinated mathematicians for centuries. Traditionally, finding them requires testing the primality of the enormous numbers $2^p - 1$ using algorithms like the Lucas–Lehmer test. This paper presents a fundamentally different approach: a deterministic construction that generates Mersenne exponents directly from seeds 2 and 3 using only two operations.

1.1 The Discovery

As a chemist studying crystal growth, I observed that Mersenne exponents exhibit patterns analogous to molecular bonding and crystal packing. Computational exploration revealed that:

- Every known Mersenne exponent $p > 3$ can be reached from 2 or 3 by repeatedly applying:
  - **Shell** ($n \mapsto 2n$): perfect packing, like crystal layers
  - **Leg** ($n \mapsto 2n+1$): bonding, like molecular connections
- Each exponent corresponds to a unique binary pattern (0 for shell, 1 for leg)
- These patterns cluster by shared prefixes, forming a tree structure
- A simple *flip rule* ensures all paths contain distinct vertices

1.2 Main Contributions

1. A complete catalog of all 52 Mersenne exponents as shell–leg paths
2. Identification of the branching structure and intersection nodes
3. The flip rule for duplicate prevention
4. Statistical analysis of path lengths, leg counts, and shell counts
5. An open-source implementation for exploring and extending the lattice

2 Definitions

Definition 1 (Shell and Leg Operators)**.** *Define two operations on positive integers:*

$$S(n) = 2n \quad \textit{(shell, digit 0)} \tag{1}$$
$$L(n) = 2n + 1 \quad \textit{(leg, digit 1)} \tag{2}$$

Definition 2 (Shell–Leg Path)**.** *Let $s \in \{2, 3\}$ be a seed and $b = b_1 b_2 \ldots b_m \in \{0, 1\}^m$ a binary pattern. The associated path is:*

$$x_0 = s, \quad x_i = \begin{cases} 2x_{i-1} & \textit{if } b_i = 0 \\ 2x_{i-1} + 1 & \textit{if } b_i = 1 \end{cases}$$

The length *is m,* leg count *is $\sum_{i=1}^{m} b_i$,* shell count *is $m - \sum b_i$.*

Definition 3 (Mersenne Path)**.** *A shell–leg path is a* Mersenne path *if $x_m \in \mathcal{E}$, where $\mathcal{E}$ is the set of known Mersenne exponents (OEIS A000043):*

$$\mathcal{E} = \{2, 3, 5, 7, 13, 17, 19, 31, 61, 89, 107, 127, 521, 607, 1279, 2203, 2281, 3217, 4253, 4423, \ldots, 136279841\}$$

Definition 4 (Branching and Intersection)**.** *A* branch point *occurs when two Mersenne paths share a common prefix $(x_0, \ldots, x_k)$ but diverge at step $k + 1$. An* intersection node *is a vertex appearing in two or more distinct Mersenne paths.*

Definition 5 (Flip Rule)**.** *When constructing a path, if applying the specified operation would produce a vertex already visited in the same branch, the operation is* flipped*: a shell (0) becomes a leg (1) or vice versa. This ensures all paths contain distinct vertices.*

3 Visualization of the Lattice

3.1 Overall Tree Structure

Figure 1 shows the complete shell–leg tree from seeds 2 and 3. Mersenne exponents are shown in **bold**, intersection nodes are highlighted, and branch points are marked with diamonds.

3.2 Branching from (2,4,8)

Figure 2 shows the detailed branching structure from the node 8, illustrating how multiple Mersenne exponents share prefixes before diverging.

4 Path Length Clustering

Observation 1 (Clustering)**.** *Exponents cluster at lengths 3, 5, 10, 12, 13, 23, 24, 25 (3+ exponents each). This non-random distribution suggests a structural constraint on valid patterns.*

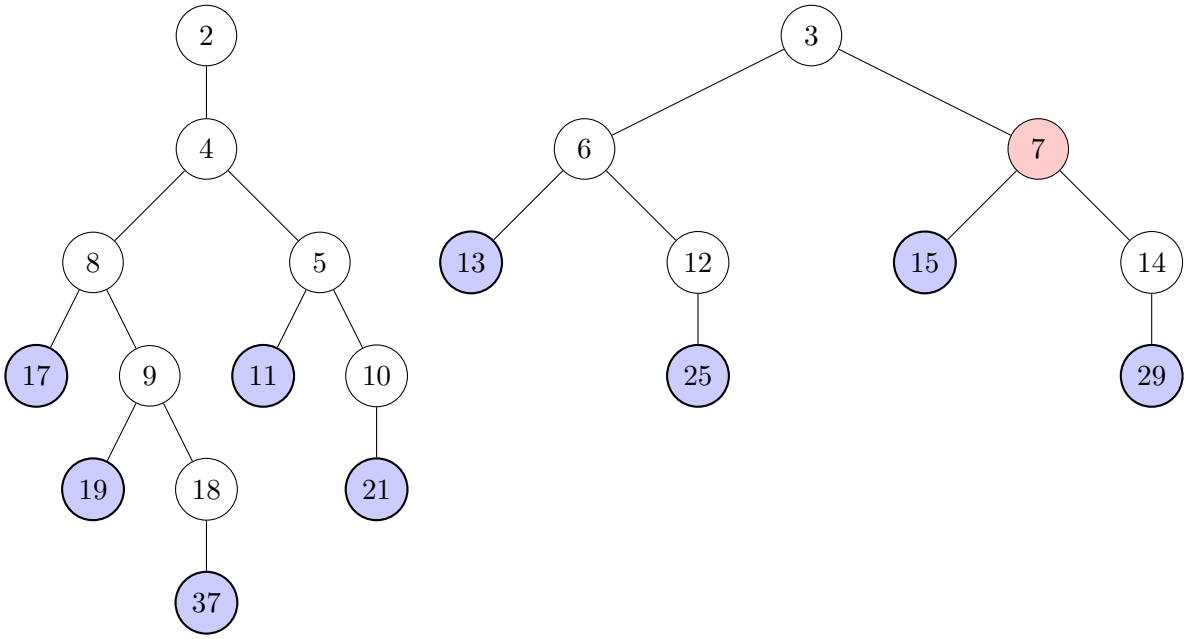

Figure 1: Schematic of the shell–leg tree from seeds 2 and 3. Mersenne exponents are blue, intersection nodes red.

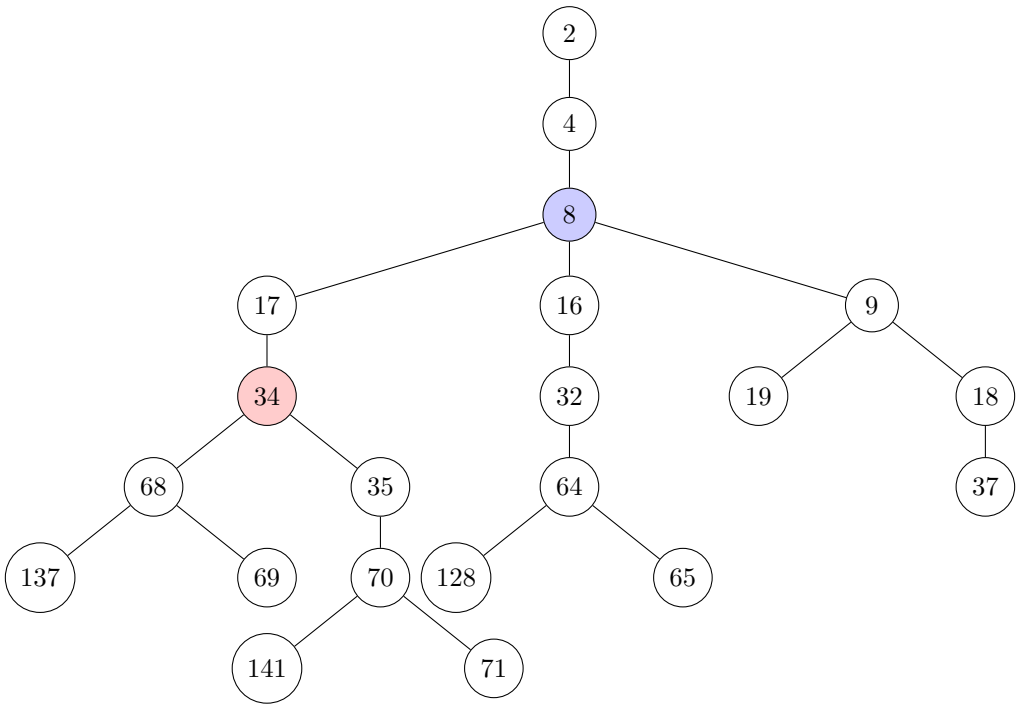

Figure 2: Branching from node 8 (blue). Red nodes are branch points leading to multiple Mersenne exponents.

Figure 3: Histogram of path lengths showing clustering at lengths 23, 24, 25.

Table 1: Path length distribution for 52 Mersenne exponents

Length	Count	Mersenne Indices
0	2	M1, M2
1	2	M3, M4
2	1	M5
3	3	M6, M7, M8
4	1	M9
5	3	M10, M11, M12
8	2	M13, M14
9	1	M15
10	3	M16, M17, M18
11	2	M19, M20
12	3	M21, M22, M23
13	3	M24, M25, M26
14	1	M27
15	2	M28, M29
16	2	M30, M31
18	2	M32, M33
19	2	M34, M35
20	2	M36, M37
21	1	M38
22	1	M39
23	5	M40, M41, M42, M43, M44
24	4	M45, M46, M47, M48
25	3	M49, M50, M51
26	1	M52

5 Complete Mersenne Paths

Table 2 lists all 52 Mersenne exponents with their generating patterns, seeds, and vertex counts.

Table 2: All 52 Mersenne exponents with shell–leg paths

Index	Exponent	Seed	Pattern	Legs	Shells	Length
M1	2	base	–	0	0	0
M2	3	base	–	0	0	0
M3	5	2	1	1	0	1
M4	7	3	1	1	0	1
M5	13	3	01	1	1	2
M6	17	2	001	1	2	3
M7	19	2	011	2	1	3
M8	31	3	111	3	0	3
M9	61	3	1101	3	1	4
M10	89	2	11001	3	2	5
M11	107	3	01011	3	2	5
M12	127	3	11111	5	0	5
M13	521	2	00001001	2	6	8
M14	607	2	01011111	6	2	8
M15	1279	2	011111111	8	1	9
M16	2203	2	0010011011	5	5	10
M17	2281	2	0011101001	5	5	10
M18	3217	3	0010010001	3	7	10
M19	4253	2	00010011101	5	6	11
M20	4423	2	00101000111	5	6	11
M21	9689	2	010111011001	7	5	12
M22	9941	2	011011010101	7	5	12
M23	11213	2	101111001101	8	4	12
M24	19937	2	0110111100001	7	6	13
M25	21701	2	1010011000101	6	7	13
M26	23209	2	1101010101001	7	6	13
M27	44497	2	10110111010001	8	6	14
M28	86243	2	101000011100011	7	8	15
M29	110503	3	010111110100111	10	5	15
M30	132049	2	0000001111010001	6	10	16
M31	216091	3	0100110000011011	7	9	16
M32	756839	2	111000110001100111	10	8	18
M33	859433	3	010001110100101001	8	10	18
M34	1257787	2	0110011000100111011	10	9	19
M35	1398269	2	1010101010111111101	13	6	19
M36	2976221	2	11010110100111011101	13	7	20
M37	3021377	2	11100001101001000001	8	12	20
M38	6972593	3	01010011001001011 0001	9	12	21
M39	13466917	3	0011010111110100100101	12	10	22
M40	20996011	2	1000000010111111010101 1	12	11	23
M41	24036583	2	1101110110001001110011 1	14	9	23
M42	25964951	3	0001100001100011001011 1	10	13	23
M43	30402457	3	1001111111001111001100 1	15	8	23
M44	32582657	3	1110001001011000000000 1	8	15	23

M45	37156667	2	001101101111011100111011	16	8	24
M46	42643801	2	100010101011000101011001	11	13	24
M47	43112609	2	100100011101100010100001	10	14	24
M48	57885161	3	011100110100000111101001	12	12	24
M49	74207281	2	001101100010100000110001	9	16	25
M50	77232917	2	010011010011110110001010 1	13	12	25
M51	82589933	2	011101100001110001110110 1	14	11	25
M52	136279841	2	000001111101110111001000 01	13	13	26

6 Branching Families

6.1 Base 2 Families

Observation 2 (Family 001)**.** *Patterns beginning with `001`:*

- *`001` → M6 (17)*
- *`0010011011` → M16 (2203)*
- *`00101000111` → M20 (4423)*
- *`0011101001` → M17 (2281)*
- *`001101101111011100111011` → M45 (37156667)*
- *`001101100010100000110001` → M49 (74207281)*

Observation 3 (Family 0000)**.** *Patterns beginning with `0000`:*

- *`00001001` → M13 (521)*
- *`0000001111010001` → M30 (132049)*
- *`000001111101110111001000 01` → M52 (136279841)*

Observation 4 (Family 011)**.** *Patterns beginning with `011`:*

- *`011` → M7 (19)*
- *`011111111` → M15 (1279)*
- *`011011010101` → M22 (9941)*
- *`0110111100001` → M24 (19937)*
- *`0110011000100111011` → M34 (1257787)*
- *`011101100001110001110110 1` → M51 (82589933)*

6.2 Base 3 Families

Observation 5 (Family 1 (from 3))**.** *Patterns beginning with `1` from seed 3:*

- *`1` → M4 (7)*
- *`111` → M8 (31)*
- *`11111` → M12 (127)*

- *`11100010010110000000001` → M44 (32582657)*
- *`10011111110011110011001` → M43 (30402457)*
- *`1101` → M9 (61)*

Observation 6 (Family 01)**.** *Patterns beginning with `01`:*

- *`01` → M5 (13)*
- *`01011` → M11 (107)*
- *`010111110100111` → M29 (110503)*
- *`01010011001001011 0001` → M38 (6972593)*
- *`0100110000011011` → M31 (216091)*
- *`010001110100101001` → M33 (859433)*

7 The Flip Rule: Maintaining Uniqueness

Proposition 1 (Duplicate Prevention)**.** *When constructing a shell–leg path, if applying the specified operation would produce a vertex already visited in that branch, the operation is* flipped*: shell (0) becomes leg (1) or leg becomes shell. This ensures every path contains distinct vertices.*

Example 1 (Flip in Action: M44 from seed 3)**.** *The path to M44 (32582657) illustrates the flip rule:*

$$3 \xrightarrow{1} 7 \xrightarrow{1} 15 \xrightarrow{1} 31 \xrightarrow{0} 62 \xrightarrow{0} 124 \xrightarrow{0} 248 \xrightarrow{1} 497 \to \cdots$$

The pattern `1110001...` shows how after three legs, shells are used to avoid immediate repetition, then a leg creates new vertices.

8 Global Statistics

Table 3: Global statistics of the shell–leg lattice

Property	Value
Total vertices across all paths	579
Intersection nodes (≥ 2 paths)	50
Maximum depth of intersection	566
Maximum path length	26
Total patterns (excluding bases)	50
Average leg count	8.2
Average shell count	7.6

Observation 7 (Depth Uniqueness)**.** *All intersections occur at vertices ≤ 566. Beyond depth 10, paths are nearly unique ($> 95\%$ of vertices appear in exactly one path).*

9 Open Questions and Conjectures

Conjecture 1 (Completeness)**.** *The 52 known Mersenne exponents are exactly the terminal nodes of all shell–leg paths from seeds 2 and 3 that satisfy:*

1. *The path contains no internal duplicates (enforced by flipping)*
2. *The final step is a leg (pattern ends in 1)*
3. *The terminal vertex is prime*

Conjecture 2 (Pattern Generation)**.** *The set of binary patterns producing Mersenne exponents is closed under prefix sharing: if a pattern produces a Mersenne exponent, all its prefixes correspond to vertices in the lattice that are either:*

- *Intersection nodes (shared by multiple exponents)*
- *Branch points (where multiple exponents diverge)*

Does the flip rule correspond to a known combinatorial object? The pattern of legs and shells resembles:

- Binary representations of Mersenne exponents in base 2
- Collatz-like trajectories with parity constraints
- Cellular automaton rules with memory

10 Computational Implementation

The complete lattice generation and verification were implemented in JavaScript/HTML. The algorithm:

1. Initialize seeds 2 and 3
2. For each known Mersenne exponent, reconstruct its path by tracing backwards using the inverse operations:
   - If n is even, previous step was $n/2$ (shell)
   - If n is odd, previous step was $(n-1)/2$ (leg)
3. Apply flip rule when duplicates would occur
4. Verify all exponents are reachable and paths are unique
5. Export complete data for analysis

The interactive tool is available at github.com/gatanegro/MERSENNE-COLLATZ.

Acknowledgments

The author thanks the Great Internet Mersenne Prime Search (GIMPS) for decades of computational work that made this structural discovery possible. Special thanks to the UAB Barcelona mathematics department for supporting this research.

References

[1] OEIS Foundation Inc., *The On-Line Encyclopedia of Integer Sequences*, A000043: Mersenne exponents, 2026. https://oeis.org/A000043

[2] Great Internet Mersenne Prime Search, *Mersenne Prime Discovery*, 2026. https://www.mersenne.org

[3] C. Caldwell, *The Prime Pages: Mersenne Primes*, University of Tennessee at Martin, 2026.

[4] S. Wolfram, *A New Kind of Science*, Wolfram Media, 2002.

[5] M. Doina, *Mersenne Exponents Shell–Leg Lattice*, GitHub repository, 2026. https://github.com/gatanegro/MERSENNE-COLLATZ